Noema × Tushe · Our Response to Your Draft

Responding to: *Tushe × Noema: Response to the Synthetic Enterprise Environment requirements*, Tushe engineering, 7 October 2026 **From:** Noema engineering **Date:** 13 October 2026 **Comes with:** *Acceptance Worksheet for Friday 16 October*, which turns everything below into a test-by-test agreement

---

1. Thank you, and where we've landed

Thank you for a thorough and candid response. You answered every section we sent, named what isn't there yet instead of smoothing it over, and pushed back with sources where you think our specification is wrong. That's exactly the working relationship we hoped for.

Our headline: we can work with what you have today, and we'd like to start this week. The capabilities we lean on hardest already exist and are running:

- a payout whose own record, the rail and the beneficiary can disagree, with nothing reconciling them;
- the clear line between "it failed" and "we don't know";
- the status query;
- seeded, checksummed, resettable scenarios with predictable IDs;
- a replayable event feed with a sequence cursor;
- deterministic idempotency conflicts;
- read-only credentials.

On your challenges: you're right on several, and we've changed our approach. Where we accept your alternative, we've written down the exact condition, so neither of us has to argue about it at acceptance.

We're also taking some work onto our side, where we can do it better ourselves (section 5). That leaves a short list of things only you can provide. **Those are essential to our build and to our first demo, and we need them within October** (section 6).

2. Our answers to your six questions (your §10)

1. Cases and disputes: out of scope for you. You're right that case handling sits closer to our product than to payments infrastructure. For this POC we'll run our own small synthetic case system, and Noema's writes go there. Disputes are not part of the POC.

A useful side effect: our credentials against your sandbox can stay **read-only for the entire POC**.

2. GT-02, the privileged answer key: we accept your counter-proposal. Your reasoning in §5.2 is sound. We'll establish what actually happened from three things:

- the checksummed scenario definition;
- your partner-administrator feed (`GET /partner/events`);
- the transition log, once its read endpoint ships.

We need two things for this to work:

- **a programmatic, read-only administrator credential** that our test harness can use without a browser session, issued to one named person on our side and never to the system under test;
- **the refusal test** proving an ordinary API key cannot reach `/partner/...` routes.

3. Scenario 19, the impossible state: we accept your replacement. You shouldn't switch off your invariants or fake state at your read layer. A cross-system contradiction is the better test, and it's achievable today. Where we want to see how our own software handles data that can't be true, we'll produce it on our side and label it as such.

4. "Material": we accept your definition and make it fully explicit. The worksheet (section 1.1) spells it out in full, with a worked example. In short: two runs with the same scenario key, version, checksum and seed must produce:

- the same objects with the same IDs;
- the same faults on the same objects in the same order;
- the same path of state changes for every object (from, to, source);
- the same final states;
- the same amounts to the kobo;
- the same events in the same order per object.

Only clock readings, durations, global feed positions and the relative order of unrelated objects may differ.

5. Concurrency: one tenant per concurrent run works for us. We expect to need about ten test tenants across developers, CI, nightly runs and demos. Can tenants be created through an API, or only by your team?

We'd like them in two batches:

- **3 to 5 this week,** by Thursday 15 October, so integration starts right away;
- **the rest by Tuesday 20 October.**

6. The P1s that matter to us. - **AUTH-02, approval states on payouts.** Real fintech flows wait on approvals, and our software has to reason about them correctly, so we need pending, approved, rejected and expired approvals on payouts. - **EVT-05 (signing)** is already done. - **GT-05 (export)** isn't needed: we'll assemble run records from your read surfaces. - **CON-01 (version tokens)** isn't needed for our write path, which now targets our own case system.

Separately, we treat **switch state as its own queryable dimension** as a P0: our specification lists the switch as an independent system (SE-01), and several of our contradiction cases depend on it.

3. Your challenges, one by one (your §5 and §6)

The fast-forward clock (§5.1): you're right

Our TIM-02 contradicts our own DATA-05. We accept per-object scheduling with on-demand advance, and handle the rest of our time-sensitive testing on our side.

AT-15's pass condition becomes: *"a scheduled transition fires at its scheduled time, and fires immediately on manual advance."*

The answer key (§5.2): agreed, with the conditions above

Grading only the final state (§5.3): agreed, and no change needed from you

You're right that final state alone can't tell "correctly escalated" apart from "did nothing". Our grading already looks at the process:

- which systems were asked;
- whether anything was re-sent while the outcome was unknown;
- whether the right records were written.

We also repeat runs to measure reliability rather than one lucky pass. Your transition log, event feed and audit trail are exactly the inputs that makes possible.

"Material" (§5.4): agreed, with the full definition in the worksheet

Scenario 19 (§5.5): agreed, as in section 2

Buying a proxy for transport faults (§6): agreed, and we'll handle it

Deliberately broken code shouldn't live in your money path. We'll run a thin proxy of our own in front of the sandbox for:

- timeouts, resets and gateway errors;
- slow responses;
- stale reads.

Commit-then-no-answer stays yours (test account `02`), for exactly the reason you gave.

Transaction Status Query (your §4.6): thank you

This is a genuinely useful addition, and we'll use it as a read when an outcome is unknown. Please confirm in writing that **calling it can never move money, reverse a debit or change a balance.** It may move a payout from unknown to a final status, but nothing more.

4. Corrections we'd like you to make

You said the draft is a working document. These are the places where it disagrees with itself or with our specification. Please look into each.

1. **The acceptance-test tally. Please look into this, because the table is what we're working from.**
   - The summary in §8 says **12 pass, 6 partial, 2 fail**.
   - The table above it shows **11 pass** (counting AT-15's "pass, differently"), **4 partial** (AT-01, 05, 14, 19) and **5 fail** (AT-04, 07, 09, 17, 20).
   - Please reconcile the summary with the table.
2. **The scenario tally.**
   - The summary says **11 executable, 5 partial, 3 requiring extension, 1 challenged**.

- The table shows **12 executable**: 1, 2, 3, 4, 5, 6, 9, 10, 12, 13, 14, 16.
- It shows **4 partial**: 7, 15, 17, 20.

3. **Requirements not covered.**
   - Our specification has **85 numbered requirements (73 P0, 12 P1)**. Your response covers 73 (63 P0, 10 P1).
   - The difference is exactly our ledger requirements, **LED-01 to LED-06 (§4.3)**, which have no rows.
   - Our six idempotency requirements, **IDP-01 to IDP-06 (§11)**, are answered in one combined row.
   - Please add rows for all twelve. Three matter most to us:
     - LED-04: one leg posted, another delayed or uncertain;
     - LED-05: integer minor units;
     - IDP-06: idempotency-key retention.
4. **"The only remaining item your acceptance tests depend on" (§1, §4.7).**
   - That describes event-delivery faults (E3).
   - Your own table also ties AT-09 and AT-14 to E6, AT-17 to E7, AT-19 to E1, and AT-20 to E5.
   - The sentence should match the table.
5. **SE-01 is marked "Supported".** Your note says the switch is not a separately queryable state. We'd describe that as "Supported differently" until switch state is queryable on its own (our request R4).
6. **TIM-03, which matters to us.**
   - The row says `delivered` doesn't exist because outbound delivery (E2) doesn't exist.
   - E2 is now marked built, and `deliveredAt` exists, per your EVT-01 row. So the TIM-03 row looks out of date.
   - Please confirm `deliveredAt` is available and update the row.
   - We rely on accurate delivery timestamps to rebuild the timeline of what happened, and to tell a late notification from a missing one.
7. **Scenarios 6 and 7.**
   - Scenario 6 (missing webhook) is mapped to a payout that never resolves. That's a different case: in a missing webhook, the state *does* change but the event never arrives.
   - Scenario 7 (out-of-order webhook) is mapped to reversal-after-completion, which is real causal order rather than out-of-order delivery.
   - Both are fine to keep as their own cases. We'll produce the delivery-side versions on our side.
8. **The indicative dates in §11 won't work for us.**
   - The dates are 24 Oct, 7 Nov, 21 Nov, 28 Nov and 12 Dec. Delivered on that schedule, they would be a major blocker for our build and for our first demo.
   - The items they cover are essential to us, so **we're asking for everything still on our list to land within October.** Section 6 gives each a date.
   - All five dates also fall on Saturdays, so working-day dates would help planning on both sides.
9. **Your name.** We've corrected it from "Tuche" to "Tushe" across all our documentation and communications. Our apologies for the error.

5. What we'll build on our side

These are built by us, so you don't need to build them for our POC:

- **Event-delivery faults (E3):** drop, delay, reorder and malformed delivery, injected on our side.
- **The stale-read profile:** our proxy returns old versions on purpose.
- **Version tokens and stale-mutation detection on our write path.** Our writes go to the case system we run, so these live there.
- **Run export (E7):** assembled from your read surfaces.
- **A case or dispute domain:** out of scope for you.
- **A privileged oracle endpoint (GT-02):** replaced by your administrator surface.
- **Transport faults:** our own proxy, so there's nothing to buy and nothing in your money path.

6. What we need from you

These are essential to our build and our first demo. The worksheet has the same list with a reason for each.

#	Request	Date
R1	**Tenants and credentials, in two batches.** Batch 1: **3 to 5 test tenants**, a read-only API key for each, one programmatic read-only administrator credential, and one test-mode payout-creating credential for building scenario histories (held by one named person on our side, never by the system under test). Batch 2: the rest, to about **10**	**Batch 1: Thu 15 Oct. Batch 2: Tue 20 Oct**
R2	**The read surfaces (E1):** transition history (STM-03), transaction history (REC-01), the correlated timeline (REC-02), causation on events (EVT-01, AUD-04), audit query by transaction, account, customer and correlation ID (AUD-05), and actor on the transition record (AUD-02, AT-19)	**No later than Fri 23 Oct**
R3	**Seeded transaction histories:** past payouts with set beneficiary names, amounts, outcomes and `occurredAt` dates, including near-duplicates	**No later than Fri 23 Oct**
R4	**Switch state** as its own queryable dimension: raw code, normalised status, and whether it is final	**No later than Fri 23 Oct**
R5	No behaviour change on the sandbox from 13 Oct to 6 Nov without 48 hours' notice, and the sandbox version on every response	**From now**
R6	Written confirmation that the status query can never move money or change balances	**Fri 16 Oct**

#	Request	Date
R7	Test-mode rate limits and quotas, and the sandbox's hosting region	**Fri 16 Oct**
R8	Answers to the questions below	**Fri 16 Oct**
R9	**Ambiguous name enquiry (E6, AT-09).** Some lookups return a single exact name; some return several possible matches; some return a name that doesn't match the account. Set by the scenario and repeatable on replay. **This is central to what we're building and is in our first demo**	**Fri 30 Oct**
R10	**Payout approval states (E6, AUTH-02):** pending, approved, rejected and expired, settable by scenario	**Fri 30 Oct**
R11	**Scenario-definition richness (the rest of E4):** adversarial fixtures, faults and event schedules declared in the definition, ordered composition of several faults, and medium and large data tiers	**Fri 30 Oct**
R12	**The diagnostics view (E5, AT-20):** what was injected, when, against which target, the configured behaviour and the actual outcome, plus a clear injected-versus-genuine-error flag. Your existing markers serve as an interim until then	**Fri 30 Oct**
R13	**Webhooks kept active** on our tenants, delivering to a public HTTPS endpoint we register, alongside the event feed	**With batch 1 (Thu 15 Oct)**

And our questions:

1. Does reset also remove payouts created through the API on a scenario's accounts?
2. Do API-created payouts in a freshly applied scenario get the same IDs every time, given the same idempotency key and reference?
3. Can test tenants be created through an API, or only by your team?
4. **Feed and webhooks.** We'll read `GET /v1/events` by sequence cursor as our main source, and keep **webhooks active** so we can test replay and deduplication. What do you need from us to register our endpoint?
5. **Integer kobo, everywhere (LED-05).** Please confirm that **every amount on every endpoint and every event is an integer number of kobo**: payouts, fees, balances, ledger entries and webhook payloads. No floats, and no decimal naira strings. Floating-point rounding on Naira amounts is exactly the kind of silent error our product exists to prevent, so this matters a great deal to us.
6. What availability should we expect between 13 October and 6 November, and who do we contact when the sandbox is down?

7. What Noema commits to

- **Our own synthetic case system**, so you never have to model case handling.
- **Our own proxy** for transport, stale-read and event-delivery faults, so no deliberately broken code goes near your money path.
- **Reading your feed by sequence as our main source,** with webhooks active alongside it.
- **Read-only use of your sandbox for the whole POC.** The one payout-creating credential is used only to build test scenarios, by one named person.
- **Showing you the tests we cover ourselves** (AT-04, 05, 07, 14 and 17) once each is complete, and sharing the evidence.
- **Reporting anything that looks like a sandbox defect** with the tenant, sandbox version, scenario, seed and the redacted request and response, so you can reproduce it before anyone fixes anything.

8. Next steps

When	What
Thu 15 Oct	Batch 1: 3 to 5 tenants and the credentials (R1), with webhooks active (R13). Our team starts integrating
Fri 16 Oct, 18:00 WAT	**One joint session:** walk through the worksheet, agree and sign it, then run the acceptance tests against it and sign the record. Answers to R6, R7 and R8
Tue 20 Oct	Batch 2: the remaining tenants (R1)
Fri 23 Oct, latest	Read surfaces (R2), seeded histories (R3), switch state (R4)
Fri 30 Oct	Ambiguous name enquiry (R9), payout approval states (R10), scenario richness (R11), diagnostics view (R12)

Thank you again. It's clear your team took our specification seriously, and we've tried to answer in the same spirit. We're looking forward to Friday.

Noema engineering